Algorithmes sur les arbres binaires de recherche

B. DARID

2026-02-18

- 1 Définition et représentation des ABR en Python
  - 1.1 Invariant d'un ABR
  - 1.2 Définition
- 2 Recherche dans un ABR
  - 2.1 Algorithme récursif
  - 2.2 Complexité en temps
  - 2.3 Une propriété magique: le parcours infixe
- 3 Insertion dans un ABR
  - 3.1 Principe général
  - 3.2 Algorithme récursif
  - 3.3 Complexité en temps
- 4 Tableau de synthèse
- 5 Encapsulation dans une classe
  - 5.1 Implémentation de la classe ABR
- 6 Parcours d'un ABR: exploiter l'ordre
  - 6.1 Les trois types de parcours récursifs
  - 6.2 Exercice d'intégration

Cadre de travail

Ce document est un **support d'activité guidée** sur les arbres binaires de recherche (ABR). Les fonctions proposées sont destinées à être **complétées par les élèves**.

Convention

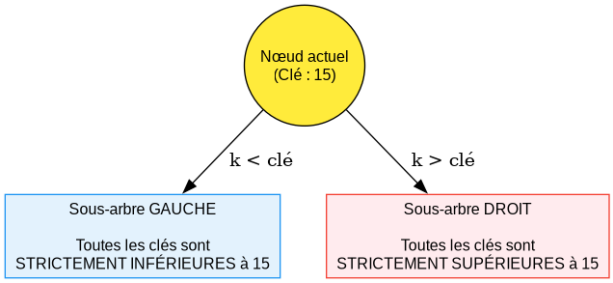
Dans la suite, on suppose que les **clés sont comparables** et, sauf mention contraire, **toutes distinctes** (pas de doublons).

1 Définition et représentation des ABR en Python

1.1 Invariant d'un ABR

- Toutes les clés du sous-arbre gauche sont **strictement inférieures** à la clé du nœud.
- Toutes les clés du sous-arbre droit sont **strictement supérieures**.
- Chaque sous-arbre est lui-même un ABR.

Pour tout nœud d'un ABR, la règle d'ordre suivante doit être respectée :



L'invariant : séparation des clés par rapport au nœud actuel

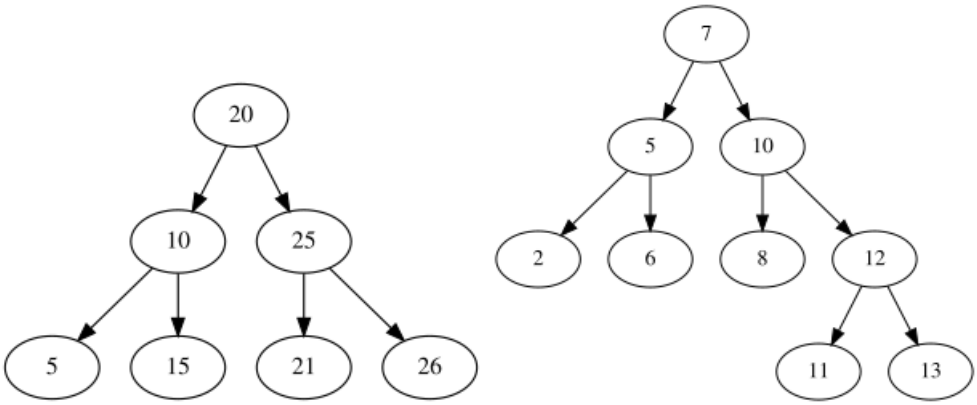
- Toutes les clés du sous-arbre gauche sont **strictement inférieures** à la clé du nœud.
- Toutes les clés du sous-arbre droit sont **strictement supérieures**.

1.2 Définition

Un arbre binaire de recherche (en abrégé **ABR**) est un arbre binaire dont les nœuds contiennent des **clés** (valeurs) comparables. Pour tout nœud d'un ABR :

- toutes les clés du sous-arbre gauche sont inférieures à celle du nœud;
- toutes les clés du sous-arbre droit sont supérieures à celle du nœud.

Exemples d'ABR



Un ABR étant un arbre binaire, on peut le représenter comme précédemment avec une classe `Noeud` avec une contrainte sur la valeur des noeuds: **ils doivent être comparables**.
Avec la classe `Noeud` du chapitre précédent, les deux ABR ci-dessus sont modélisés de la façon suivante:

```
class Noeud:
    """ Une classe pour modéliser les noeuds d'un arbre"""

    def __init__(self, v, fg=None, fd=None):
        self.val = v
        self.gauche = fg
        self.droit = fd
```

```
abr1 = Noeud(20,
            Noeud(10, Noeud(5), Noeud(15)),
            Noeud(25, Noeud(21), Noeud(26))
            )

abr2 = Noeud(7,
            Noeud(5, Noeud(2), Noeud(6)),
            Noeud(10, Noeud(8), Noeud(12, Noeud(11), Noeud(13)))
            )
```

Note

- Dans la méthode `__init__`, `fg` et `fd` ont une valeur par défaut égale à `None`, cela permet juste une écriture plus compacte lors de la création d'une instance en évitant de réécrire `None` lorsqu'un noeud n'a pas de fils.
- Dans le code, la clé stockée dans un nœud est contenue dans l'attribut `val`.

2 Recherche dans un ABR

Chercher une clé dans un ABR revient à **la comparer à la clé de la racine** puis, si elle est différente, à poursuivre la recherche dans **un seul** des sous-arbres.

La recherche d'une clé x dans un arbre binaire de recherche peut être menée avec l'algorithme ci-dessous qui renvoie `Vrai` ou `Faux` selon que la recherche aboutit ou pas.

2.1 Algorithme récursif

```
FONCTION rechercher(noeud, x) :

Entrées : x (la clé recherchée), noeud (racine d'un arbre binaire de recherche)
Sortie : Booléen

SI noeud est vide:
    Renvoyer Faux
SI x = Clé(noeud):
    Renvoyer Vrai
Sinon SI x < Clé(noeud):
    Renvoyer rechercher(x, fils_gauche(noeud))
Sinon
    Renvoyer rechercher(x, fils_droit(noeud))
```

Exercice

Implémenter cet algorithme en python.

```python
def rechercher(noeud, x):
    """
    Recherche la clé x dans l'arbre dont la racine est 'noeud' et
    renvoie un booléen indiquant la présence ou non de x dans l'arbre.
    x : clé comparable
    noeud : racine (Noeud) ou None
    """

    if noeud is None:
        return False
    # À compléter en suivant le pseudo code
```

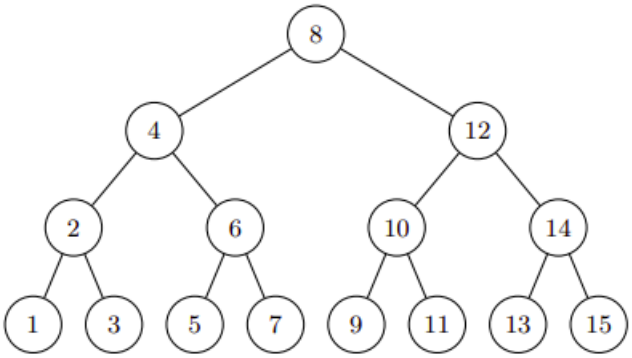
```python
# À exécuter
assert rechercher(abr2, 10) == True
assert rechercher(abr2, 35) == False
print("Validé !")
```

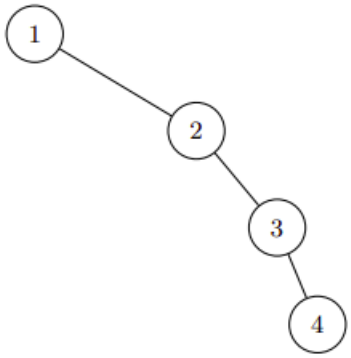
2.2 Complexité en temps

> **Note**
>
> - Si l'arbre est équilibré, les opérations de recherche et d'insertion sont en **O(log n)**.
> - Dans le pire des cas (arbre dégénéré), elles deviennent **linéaires**.

Dans le cas idéal, **si l'arbre est équilibré**, chaque appel de `rechercher(arbre, x)` élimine la moitié des éléments. Cette recherche est alors à rapprocher de la recherche dichotomique vue en 1re, dont la complexité est **logarithmique**.
Dans le pire des cas, si l'arbre est *filiforme*, la recherche est peu efficace et sa complexité est alors linéaire.

Illustration





Arbre binaire de recherche complet
Hauteur $h \approx \log_2(n)$
Complexité de recherche : $O(\log n)$

Arbre binaire de recherche dégénéré
Hauteur $h \approx n$
Complexité de recherche : $O(n)$

> Dans un Arbre Binaire de Recherche, **le nombre de comparaisons effectuées pour trouver un élément (ou constater son absence) est au maximum égal à la hauteur de l'arbre**.
>
> Dans l'arbre équilibré (gauche): la structure est optimisée, la hauteur est minimale.Pour $n = 15$ nœuds, la hauteur est $h = 4$. On ne fera jamais plus de 4 comparaisons. On dit que $h \approx \log_2(n)$.
> Dans l'arbre dégénéré/filiforme (droite): l'arbre ressemble à une liste chaînée, la hauteur est maximale. Pour $n = 4$ nœuds, la hauteur est $h = 4$. On peut monter jusqu'à 4 comparaisons.On dit que $h \approx n$.
>
> À retenir : chercher dans un arbre équilibré de 1 million d'éléments ne prend que 20 comparaisons ($2^{20} \approx 10^6$), alors que dans un arbre filiforme, cela peut en prendre 1 million.

2.3 Une propriété magique: le parcours infixe

Il existe une relation directe entre la structure de l'ABR et l'ordre des éléments.

Propriété

> Le parcours infixe (gauche-racine-droite) d'un Arbre Binaire de Recherche visite les nœuds dans l'ordre croissant de leurs clé.

Exemple

Sur le schéma ci-dessus, si on applique le parcours infixe à l'arbre de gauche:

$$1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15$$

On retrouve exactement la liste triée.

3 Insertion dans un ABR

On veut insérer une clé `k` dans un arbre binaire de recherche en **préservant l'invariant**.

3.1 Principe général

Pour insérer une clé `k` dans un ABR, on compare `k` à la clé du nœud courant :

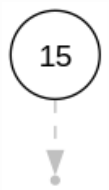
- si l'arbre est vide, on crée un nouveau nœud contenant la clé `k` (cas de base);
- sinon, on compare `k` à la clé du nœud courant, puis on construit l'arbre obtenu en insérant récursivement `k` dans le sous-arbre gauche ou droit approprié.

L'algorithme doit préserver la propriété d'ordre des arbres binaires de recherche.

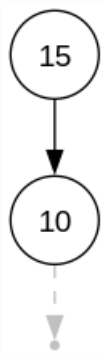
3.2 Algorithme récursif

```
Fonction inserer(nœud_actuel, k)
Entrées:
    k: une clé à insérer
    noeud_actuel: un objet Noeud
Sortie: le noeud actuel dans lequel k a été inséré

Si nœud_actuel est vide:
    On a trouvé l'emplacement, renvoyer un nouveau Nœud(k)

Sinon:
    Si k < clé(nœud_actuel):
        Le fils gauche s'occupe de l'insertion. Le nouveau fils gauche devient:
        fils_gauche(nœud_actuel) = inserer(fils_gauche(nœud_actuel), k)

    Sinon si k > clé(nœud_actuel) :
        Le fils droit s'occupe de l'insertion. Le nouveau fils droit devient:
        fils_droit(nœud_actuel) = inserer(fils_droit(nœud_actuel), k)

    Renvoyer nœud_actuel au parent (sous-arbre mis à jour)
```

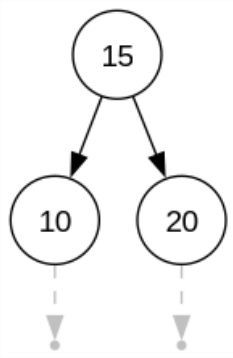
Déroulement de l'algorithme sur la liste de clé [15, 10, 20, 8, 12]



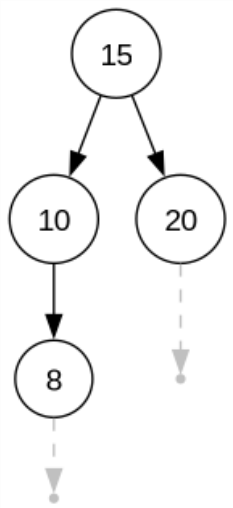
Étape 0 : On insère 15 à la racine vide.



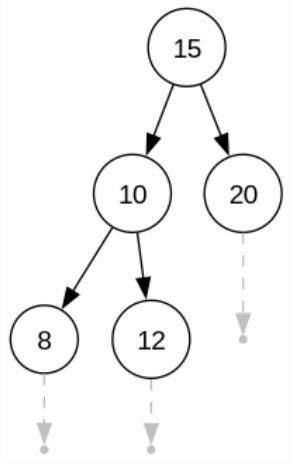
Étape 1 : $10 < 15$, insertion à gauche.



Étape 2 : $20 > 15$, insertion à droite.



Étape 3 : $8 < 10$, on descend à gauche de 10.



Étape 4 : $12 > 10$, on descend à droite de 10.

Exercice

Implémenter cet algorithme en Python.

```python
def inserer(noeud, k):
    """
    Insère la clé k dans l'arbre dont la racine est 'noeud' et renvoie l'arbre mis à jour.
    k : clé comparable avec les clés présentes dans l'arbre
    noeud : noeud courant ou None
    """
    if noeud is None:
        return Noeud(k)
    # À compléter en suivant le pseudo code
```

3.3 Complexité en temps

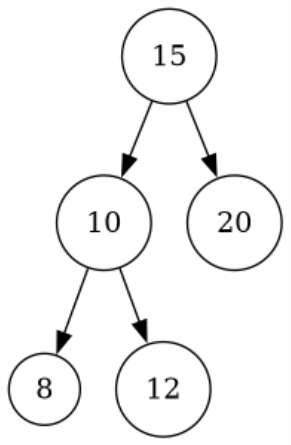
L'insertion *coûte* la même chose que **la recherche**. En effet, insérer, c'est d'abord chercher !
Pour insérer une valeur, on parcourt l'arbre depuis la racine (comme pour une recherche). On descend à gauche ou à droite selon la valeur. On s'arrête dès qu'on trouve une place vide (un fils None) pour y placer le nouveau nœud.

On parcourt un chemin de la racine vers une feuille, le nombre de comparaisons est, là aussi, borné par la hauteur h de l'arbre.

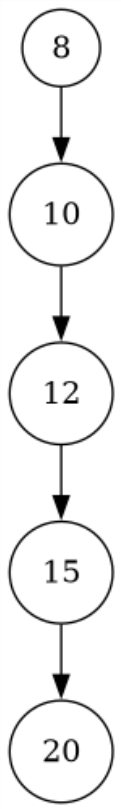
4 Tableau de synthèse

	Cas équilibré	Cas filiforme
Recherche	O(log n)	O(n)
Insertion	O(log n)	O(n)
Hauteur *h*	≈ $\log_2(n)$	≈ n

Illustration



Cas 1 : Arbre équilibré
(h=3).



Cas 2 : Arbre dégénéré
(h=5).

Justifications

- **Recherche** : On descend d'un niveau à chaque étape.
- **Insertion** : Insérer revient à chercher la place d'une feuille.
- **Hauteur** : La performance dépend de la "forme" de l'arbre.

5 Encapsulation dans une classe

Jusqu'à présent, nous avons manipulé des nœuds et des fonctions externes. C'est le moteur de notre programme. Cependant, pour un utilisateur final, manipuler des `None` ou devoir gérer la racine manuellement n'est pas idéale.

Nous allons utiliser la classe ABR comme une interface sécurisée. Elle possède un seul attribut, `self.racine`, et utilise les fonctions récursives que nous avons créées.

5.1 Implémentation de la classe ABR

Consigne: complétez les méthodes de la classe ABR ci-dessous.

La méthode `chercher` doit renvoyer le résultat de la fonction `rechercher(noeud, ...)` vue en section 2.

La méthode `ajouter` doit mettre à jour l'attribut `self.racine` en utilisant la fonction `inserer(noeud, ...)` vue en section 3.

Exercice

```
class ABR:
    def __init__(self):
        """Initialise un arbre vide (la racine ne pointe vers rien)"""
        self.racine = None

    def chercher(self, x):
        """Méthode d'interface pour la recherche"""
        # À compléter (utiliser la fonction rechercher)
        pass

    def ajouter(self, x):
        """Méthode d'interface pour l'insertion"""
        # À compléter (utiliser la fonction inserer)
        pass
```

6 Parcours d'un ABR: exploiter l'ordre

Une fois l'arbre construit, comment accéder à ses données ? Contrairement à une liste, un arbre peut se parcourir de plusieurs façons selon l'ordre dans lequel on visite la racine et ses enfants (*sous-arbres*).

6.1 Les trois types de parcours récursifs

Pour chaque nœud, on considère trois étapes : G (sous-arbre Gauche), D (sous-arbre Droit) et E (Étiquette/Clé du nœud).

- **Parcours préfixe (E-G-D)**: on affiche la clé, puis on va à gauche, puis à droite.
- **Parcours infixe (G-E-D)**: on va à gauche, on affiche la clé, puis on va à droite. **C'est celui qui permet d'afficher les clés d'un ABR dans l'ordre croissant**.
- **Parcours postfixe (G-D-E)**: on visite les enfants avant d'afficher la clé.

6.2 Exercice d'intégration

Consigne : ajoutez une méthode afficher à la classe `ABR` qui utilise un parcours infixe pour afficher les clés de l'arbre.

Exercice

```python
def afficher(self):
    """Affiche le contenu de l'arbre par ordre croissant"""

    self._parcours_infixe(self.racine)
    print() # Pour le retour à la ligne

def _parcours_infixe(self, n):
    """Méthode interne récursive"""

    if n is not None:
        # À compléter : appliquez la logique G-E-D
        pass
```